Part A: Regex Fundamentals (Foundation)

A1. What is Regex?

Regex (Regular Expression) is a special pattern language. It is used to find specific pieces of text, such as emails, phone numbers, or dates, inside a big block of text.

Why it matters: Raw text (emails, chats, documents) contains useful information mixed with a lot of other text. Without regex, we would have to search everything by hand, which is slow and full of mistakes.

Where it is used in NLP pipeline: Regex is used in the preprocessing step. Before any NLP model works on the data, we clean the text or pull out specific information (like emails or phone numbers) using regex.

A2. Character Classes

Character classes tell regex what type of character to look for.

Symbol	Meaning
\d	Any digit (0-9)
\w	Any word character (letters, digits, underscore)
\s	Space, tab, or new line
[a-z]	Any lowercase letter from a to z
[A-Z0-9]	A custom set - capital letters or digits

Why it matters: These are the basic building blocks of regex. Without them, no pattern can be written.

Where it is used in NLP pipeline: Every extraction task (email, phone number, date) is built using these character classes.

A3. Quantifiers

Quantifiers tell regex how many times a character or group can repeat.

Symbol	Meaning
*	Zero or more times
+	One or more times
?	Zero or one time (optional)
{n,m}	At least n times, at most m times

Why it matters: In real text, patterns do not always have a fixed length. For example, a phone number can have 10 or 11 digits. Quantifiers handle this flexibility.

Where it is used in NLP pipeline: Quantifiers help match items whose length can change, such as order IDs and dates.

A4. Groups ( )

Groups are used to capture one specific part of a match, so we can pull it out separately later.

Example: (\d{3})-(\d{4}) will separately capture the area code and the number.

Why it matters: Sometimes we do not need the whole match - we only need one small part of it (for example, only the year from a date).

Where it is used in NLP pipeline: Groups are used when building structured data, such as creating a CSV or JSON file where each field must be separated.

Part B: Pattern-Based Extraction (using re.findall())

Concept: re.findall(pattern, text) finds all matches of a pattern in a piece of text and returns them as a list.

B1. Email Extraction

```
import re
pattern = r'[\w.-]+@[\w-]+\.[a-zA-Z]{2,}'
re.findall(pattern, text)
```

Why it matters: We often need to automatically find email addresses, for example for contact lists or spam filtering.

Where it is used in NLP pipeline: This is part of information extraction and data cleaning (for example, hiding personal information).

B2. Phone Number Extraction

```
pattern = r'\+?\d{0,2}[-.\s]?\d{3}[-.\s]?\d{3}[-.\s]?\d{4}'
```

Why it matters: Phone numbers appear in many formats (with dashes, spaces, or country codes). This pattern handles those different formats.

Where it is used in NLP pipeline: Used in customer support systems and CRM (customer data) tools to pull out contact numbers.

B3. Order ID Extraction

Example format: ORD-2024-00123

```
pattern = r'ORD-\d{4}-\d{5}'
```

Why it matters: Business emails and documents often contain specific IDs that need to be pulled out automatically.

Where it is used in NLP pipeline: Used for linking records and matching data in a database.

B4. Date Extraction

```
pattern = r'\d{1,2}[/-]\d{1,2}[/-]\d{2,4}'
```

Why it matters: Dates appear everywhere, but in many different formats. Extracting them is the first step before we can standardize them.

Where it is used in NLP pipeline: Used in timeline analysis and event extraction tasks.

B5. Percentage Extraction

```
pattern = r'\d+(\.\d+)?%'
```

Why it matters: We often need to pull numeric facts (like statistics or percentages) out of reports and articles.

Where it is used in NLP pipeline: Used in data analytics and NLP tasks based on numbers or statistics.

Part C: Word Tokenization

C1. What is Tokenization?

Tokenization means breaking text into small pieces called tokens. Tokens are usually words and punctuation marks.

Example: "I don't like it." becomes ["I", "do", "n't", "like", "it", "."]

Why it matters: No NLP model can understand raw sentences directly. It needs tokens, which can later be turned into numbers for the model.

Where it is used in NLP pipeline: Tokenization is the first step of almost every NLP pipeline. Everything else, like POS tagging, parsing, or creating embeddings, happens after this step.

C2. NLTK Tokenization

```
from nltk.tokenize import word_tokenize
word_tokenize("I don't like it.")
```

NLTK tokenization is simple, rule-based, and fast.

C3. spaCy Tokenization

```
import spacy
nlp = spacy.blank("en")
doc = nlp("I don't like it.")
[token.text for token in doc]
```

spaCy tokenization is smarter and follows more detailed language rules.

C4. NLTK vs spaCy - Comparison

Feature	NLTK	spaCy
Speed	Medium	Fast
Contractions (don't)	Splits differently	Handles smartly
Output type	List of plain text words	Token objects (with extra attributes)
Best used for	Learning and research	Real-world / production NLP projects

Why it matters: The two tools can give different tokenization results. Students should know which tool to choose in a real project.

Where it is used in NLP pipeline: The choice of tokenizer affects the whole pipeline, because every later step (like NER or parsing) depends on how the text was tokenized.

Part E: Token Attributes

In spaCy, every token is an object. This object already has built-in attributes that tell us what 'type' of token it is.

Attribute	What it tells us
token.is_alpha	Is the token made only of letters?
token.like_num	Does the token look like a number (digits or number words)?
token.like_email	Does the token look like an email address?
token.is_currency	Is the token a currency symbol (like $ or Rs)?

```
for token in doc:
    print(token.text, token.is_alpha, token.like_num, token.like_email,
token.is_currency)
```

Why it matters: Instead of writing our own regex every time, spaCy already tells us what category a token belongs to. This saves time and improves accuracy.

Where it is used in NLP pipeline: These attributes are directly used for rule-based filtering, data cleaning, and feature engineering - for example, separating numbers, hiding emails, or highlighting currency values.

Part F: Sentence Segmentation

F1. What is Sentence Segmentation?

Sentence segmentation means breaking a paragraph into individual sentences.

```
import spacy
nlp = spacy.load("en_core_web_sm")
doc = nlp("Dr. Ali went to the market. He bought fruits.")
for sent in doc.sents:
    print(sent.text)
```

Why it matters: Many NLP tasks (summarization, translation, question answering) work at the sentence level. Processing a whole paragraph at once can give wrong results.

Where it is used in NLP pipeline: This step comes after tokenization and before higher-level tasks like parsing, summarization, or named entity recognition (NER). It is a necessary middle step in the pipeline.

F2. Tricky Cases

- Abbreviations like "Dr.", "e.g.", "Mr." can be mistaken for the end of a sentence.
- spaCy's trained model (not the blank pipeline) handles these tricky cases better, because it has learned real language patterns.

Why it matters: If sentence boundaries are detected wrongly, every later step (summarization, sentiment analysis, translation) can also go wrong.

Where it is used in NLP pipeline: Good sentence segmentation is the foundation for document-level NLP applications, such as chatbots and text summarizers.

Quick Recap Table (Pipeline Order)

Step	Topic	Pipeline Stage
1	Regex Fundamentals	Preprocessing / cleaning
2	Pattern-Based Extraction	Information extraction
3	Word Tokenization	Text into tokens
4	Token Attributes	Feature engineering / filtering
5	Sentence Segmentation	Tokens into sentence-level structure

This order matters because in a real NLP system, text is first cleaned and important information is extracted, then it is tokenized, then we check the properties of each token, and finally the text is organized into sentences. After this, later steps like POS tagging, NER, or model training can begin.